co-Suite Architecture Brief

Date: 2026-06-03

Architecture Goal

الهدف المعماري هو أن كل شيء يدور حول platform غني بالميزات إلى prototype يجب أن يتطور من co-Suite منظم وقابل للتوسع. Suite مستقلة بدل منطق موزع داخل صفحات workflows/jobs واضحة، وأن تكون عمليات التوليد، النشر، الحملات، التحليلات، والجدولة مبنية كـ Memory الواجهة.

المطلوبة من منظور عملي: ما الذي نملك اليوم، ما الحدود التي يجب تثبيتها، وما المسار الهندسي الذي يجعل المنتج يكبر بدون architecture هذه الوثيقة تشرح أن يصبح fragile.

Current High-Level Shape

النظام الحالي تقريبًا:

```
Next.js Web App
  -> API Client
  -> FastAPI Backend
      -> Auth / Suites / Onboarding / Content / Connections / Analytics / Billing / Product Bulk
      -> Services: brand_ai, content_generator, product_bulk_generator, publisher, analytics, google_ads, meta_oauth
      -> DB Models: User, Suite, ContentPost, GenerationJob, ProductBulk*
      -> Media Storage: R2 or local fallback
      -> AI Providers: OpenAI / Anthropic / Google image/video models
      -> External Platforms: Meta, Google Ads
```

هذا جيد كبداية. المشكلة ليست وجود الخدمات، بل أن بعض المسؤوليات ما زالت متداخلة:

- صفحات الواجهة تحمل منطق كثير.
- `brand` داخل `Suite` يستخدم كـ JSON كبير ومتوسع.
- generation flows متعددة لكنها تحتاج contract موحد.
- AI feedback/rules غير مفصول كذاكرة قابلة للإدارة.
- integrations فيها read/write/publish/analytics permissions مختلطة.

Core Architectural Principle

Suite Memory is the System of Record

كل Suite يجب أن يملك ذاكرة تسويقية منظمة. هذه الذاكرة هي المصدر الرسمي لكل generation/campaign/calendar decision.

Suite Memory تشمل:

- Business Profile.
- Brand Profile.
- Audience Profile.
- Language Profile.
- Content Rules.
- Visual Assets.
- Personas.
- Products/Services.
- Campaign Preferences.
- Feedback Learnings.
- Platform Connections Summary.

`SuiteContextBuilder` يبني context واحد واضح بدل أن كل service يقرأ `suite.brand` بطريقته. أي AI generation يجب أن يمر عبر.

Recommended Domain Boundaries

1. Identity and Account Domain

Responsibilities:

- Users.
- Auth.
- Account type: business / creator / agency.
- Billing identity.
- Team/seats لاحقًا.

Current files:

- `api/routers/auth.py`
- `api/models/user.py`
- `web/src/store/auth.ts`

Architecture direction:

- إبقاء هذا النطاق بسيطًا.
- لا نضع business/brand decisions داخل User.
- agency support لاحقًا يحتاج Account/Workspace أعلى من Suite.

2. Suite Domain

Responsibilities:

- Suite lifecycle.
- Suite status.
- Suite ownership.
- Suite navigation/workspace.
- Suite Memory root.

Current files:

- `api/models/suite.py`
- `api/routers/suites.py`
- `web/src/app/(dashboard)/suite/*`

Architecture direction:

- Suite هو aggregate root.
- لا يجب أن تصبح `brand` JSON غير محدود بدون structure.
- المرحلة القادمة: تقسيم brand JSON منطقيًا حتى لو بقي مخزنًا كـ JSONB:

```
{
  "business_profile": {},
  "audience_profile": {},
  "brand_profile": {},
  "language_profile": {},
  "content_rules": {},
  "visual_assets": {},
  "personas": [],
  "feedback_rules": []
}
```

يمكن تنفيذ هذا تدريجيًا مع backward compatibility.

3. Onboarding and Intelligence Domain

Responsibilities:

- Read website/social links.
- Gather public intelligence.
- Extract business profile.
- Suggest audience/brand/services.

- Save step-by-step onboarding.
- Regenerate profile sections.

Current files:

- `api/routers/onboarding.py`
- `api/services/brand_ai.py`
- `api/services/multi_scraper.py`
- `api/services/strategy_generator.py`
- `web/src/app/(dashboard)/suite/new/page.tsx`

Architecture direction:

Create clear services:

```
SourceGatherer
  -> WebsiteScraper
  -> SocialScraper
  -> SearchScraper

BrandExtractor
  -> takes gathered sources
  -> outputs structured Suite Memory patch

AudienceSuggestionService
  -> creates interests / behaviors / demographics / notes

BrandAssetAnalyzer
  -> classifies logos, fonts, colors, images

OnboardingStepService
  -> validates and saves each step
```

Important:

- AI suggestions must be regeneratable per section.
- User edits override AI.
- Each step should know whether data is `user_confirmed`, `ai_suggested`, or `inferred`.

4. Generation Domain

Responsibilities:

- Ideas.
- Copy.

- Images.
- Carousels.
- Videos.
- Product bulk media.
- Regeneration with feedback.
- Provider limits and queues.

Current files:

- `api/routers/content.py`
- `api/services/content_generator.py`
- `api/services/product_bulk_generator.py`
- `api/services/generation_jobs.py`
- `api/models/generation_job.py`
- `api/models/content.py`

Architecture direction:

Generation should be job-based, not request-based.

Recommended flow:

```
CreateGenerationRequest
  -> validate suite and billing
  -> build SuiteContext
  -> create GenerationJob
  -> enqueue work
  -> worker executes steps
  -> writes ContentPost/ProductBulkAsset
  -> updates GenerationJob progress
```

Required entities:

```
GenerationJob
  id
  suite_id
  type
  status
  provider
  progress
  input_payload
  result_payload
  error
  retry_at
  created_at
  updated_at

GenerationArtifact
  id
  job_id
  suite_id
  type: image/video/carousel/text/product_asset
  media_url
  metadata
```

Today `ContentPost` and `ProductBulkAsset` cover much of this. The gap is unifying job/artifact handling across all generation types.

5. Content Domain

Responsibilities:

- Generated content lifecycle.
- Review.
- Edit.
- Approve/reject.
- Regenerate.
- Schedule.
- Publish.
- Mark used externally.

Current files:

- `api/models/content.py`
- `api/routers/content.py`
- `api/services/publisher.py`
- `web/src/components/suite/SuiteLegacyDashboard.tsx`

Architecture direction:

ContentPost should be the canonical user-facing content item.

Status flow:

```
draft/generated -> pending -> approved -> scheduled -> published
                           -> rejected -> regenerating -> pending
                           -> used_externally
```

Every rejection should create feedback:

```
ContentFeedback
  suite_id
  post_id
  feedback_text
  feedback_type
  converted_to_rule: bool
```

Later, feedback becomes `SuiteMemory.feedback_rules`.

6. Publishing and Scheduling Domain

Responsibilities:

- Schedule posts.
- Publish to Meta/Instagram/Facebook.
- Later TikTok/LinkedIn/WordPress.
- Track publishing results.
- Retry failures.

Current files:

- `api/services/publisher.py`
- `api/engine/scheduler.py`
- `api/engine/meta_publisher.py`

Architecture direction:

Separate publishing from generation completely.

Recommended entities:

```
PublishJob
  content_post_id
  suite_id
  platforms
  status
  scheduled_at
  published_at
  result
  error
```

Publishing should never depend on local file URLs. R2/public storage is required for media publishing.

7. Integrations Domain

Responsibilities:

- Meta OAuth.
- Google OAuth.
- Connection status.
- Permissions.
- Tokens.
- External account selection.

Current files:

- `api/routers/connections.py`
- `api/services/meta_oauth.py`
- `api/services/google_ads.py`
- `api/services/meta_ads_manager.py`

Architecture direction:

Connections should be provider adapters:

```
IntegrationProvider
  get_auth_url()
  handle_callback()
  list_accounts()
  select_account()
  get_connection_status()
  fetch_analytics()
  fetch_campaigns()
  publish()
```

Do not mix “connection exists” with “permission supports analytics/campaigns/publishing”. A connection can be partial.

Connection status should include:

- auth_connected.
- selected_account.
- publishing_ready.
- analytics_ready.
- ads_read_ready.
- ads_write_ready.
- missing_permissions.

8. Campaign Domain

Responsibilities:

- Read active campaigns.
- Show campaign/ad set/ad metrics.
- Build campaign drafts.
- Later launch, pause, edit.

Current files:

- `api/services/meta_ads_manager.py`
- `api/services/google_ads.py`
- `web/src/app/(dashboard)/suite/[id]/campaigns/page.tsx`

Architecture direction:

Campaign Builder must start as draft-first.

Recommended flow:

```
CampaignBrief
  -> AI suggests objective/audience/budget/creative set
  -> user reviews draft
  -> platform-specific plan generated
  -> optional launch after permissions and confirmation
```

Never launch paid campaigns automatically without explicit user confirmation.

9. Analytics Domain

Responsibilities:

- Page metrics.
- Campaign metrics.
- Content performance.
- Warnings about permissions.
- Time windows.

Current files:

- `api/routers/analytics.py`
- `api/services/analytics.py`
- `web/src/app/(dashboard)/suite/[id]/analytics/page.tsx`

Architecture direction:

Analytics must be explicit about data quality:

- real value.
- unavailable due to permissions.
- unavailable due to no connection.
- unavailable due to platform API limitation.

Do not show zeros when the real state is “permission missing”.

10. Product Bulk Domain

Responsibilities:

- Excel parsing.
- ZIP upload.
- Image matching.
- First product template generation.
- Template approval.
- Full catalog generation.
- Per-asset review.

Current files:

- `api/models/product_bulk.py`
- `api/routers/product_bulk.py`
- `api/services/product_bulk_parser.py`
- `api/services/product_bulk_generator.py`
- `web/src/app/(dashboard)/suite/[id]/product-bulk/page.tsx`

Architecture direction:

This is a separate workflow, not just content generation.

Keep it separate but reuse:

- SuiteContextBuilder.
- MediaStorageService.
- GenerationJob.
- Feedback rules.

11. Media and Asset Storage

Responsibilities:

- Generated media.
- Uploaded logos/fonts/personas.
- Product images.
- Public URLs for publishing.

Current files:

- `api/services/media_storage.py`
- R2 config.
- local fallback.

Architecture direction:

R2 should be production default. Local fallback is only for development.

Asset categories:

```
brand/logo
brand/font
brand/persona
generated/content
generated/product_bulk
uploads/product_images
```

Every stored asset should have:

- url.
- storage key.
- content type.
- source.

- dimensions when image/video.
- linked suite_id.

Frontend Architecture

Current frontend uses Next.js App Router. Direction is good, but components need clearer separation.

Recommended frontend layers:

```
app routes
  -> page composition only

components/suite
  -> suite workspace components

components/onboarding
  -> onboarding wizard sections

components/content
  -> content cards/review/publish controls

components/forms
  -> reusable inputs/chips/sections

lib/api.ts
  -> typed API client

lib/i18n
  -> translations and language context
```

Critical frontend rule:

Pages should not contain large business logic. They should compose components and call hooks.

Example:

```
suite/new/page.tsx
  currently too large
  should become:
    NewSuiteWizard
    AudienceStep
    BrandStep
    PersonaStep
    ServicesStep
```

AI Architecture

AI should be provider-agnostic at the orchestration level.

Recommended layers:

```
AIProviderClient
  OpenAI
  Anthropic
  Google

PromptBuilder
  onboarding prompts
  content prompts
  image prompts
  video prompts
  campaign prompts

SuiteContextBuilder
  normalized context for generation

GenerationOrchestrator
  decides steps and provider

QualityGuard
  checks language consistency, missing assets, dimensions, text rules
```

Provider choices:

- OpenAI: onboarding/profile building and text reasoning tests.
- Claude/Anthropic: idea/copy generation where currently strong.
- Google image/video: image/video generation, especially when multilingual text rendering is needed.

Important: model selection should be a setting, not hardcoded inside UI.

Data Model Direction

Current important models:

- User.
- Suite.
- ContentPost.
- GenerationJob.
- ProductBulkBatch.
- ProductBulkItem.

- ProductBulkAsset.
- Billing models.

Recommended additions over time:

```
SuiteMemoryRevision
ContentFeedback
PublishJob
CampaignDraft
CampaignSyncSnapshot
IntegrationConnection
StoredAsset
CalendarPlan
CalendarItem
LanguageRule
```

Do not add all at once. Add when the feature needs durable state.

Queue and Scalability

Generation cannot stay synchronous.

Required direction:

- API creates job.
- Worker processes job.
- UI polls or subscribes to job status.
- Provider limits create `waiting_provider_limit`.
- Long video/image jobs do not block API request.

Current GenerationJob is a good start. Next step is making all expensive work use it consistently.

For production, consider:

- Redis/RQ/Celery/Arq or managed queue.
- Railway worker service.
- DB-backed queue short-term if traffic is low.

Security and Compliance

Key risks:

- Social/ad tokens.
- Publishing permissions.

- AI-generated content liability.
- Uploaded brand assets.
- Campaign spending.
- User data from websites/social profiles.

Architecture requirements:

- Encrypt or securely store platform tokens.
- Scope OAuth permissions tightly.
- Explicit confirmation before paid campaign launch.
- Legal pages cover AI usage and platform connections.
- Audit log for publish/campaign actions.
- Never expose tokens to frontend.

Mobile and Future Apps

Current web must be mobile-friendly, but native apps later should not require rewriting business logic.

Architecture direction:

- Backend API remains canonical.
- Web and mobile use same API.
- Push notifications later via notification service.
- Mobile app focuses on review/approve/alerts rather than full complex setup first.

First mobile app use cases:

- Approve/reject content.
- View generated media.
- Receive generation finished notification.
- Approve scheduled posts.
- Campaign/connection warnings.

Deployment Architecture

Current direction:

- Railway web service.
- Railway API service.
- Database.
- R2 media.
- GitHub-based deploys.

Recommended service split:

```
web service
api service
worker service
database
object storage
```

Do not put long generation work only in web/API request lifecycle.

Observability

Required logs/metrics:

- generation job status and duration.
- provider failures.
- provider limit waits.
- publish failures.
- OAuth permission failures.
- media storage failures.
- campaign sync failures.

User-facing errors should be translated into actionable messages.

Example:

Bad:

```
Meta API 400
```

Good:

```
Meta analytics permission missing. Reconnect Facebook with pages_read_engagement
or continue without analytics.
```

Near-Term Refactor Plan

Phase 1: Stabilize Suite Memory

- Normalize brand JSON into sections.

- Add helper `build_suite_context(suite)`.
- Make all generation use it.
- Add source metadata: user_confirmed / ai_suggested / inferred.

Phase 2: Split Onboarding Components

- Extract AudienceStep.
- Extract BrandStep.
- Extract PersonaStep.
- Extract ServicesStep.
- Make mobile UI clean.

Phase 3: Unify Generation Jobs

- Make content generation, product bulk, regenerate, video generation use consistent GenerationJob lifecycle.
- Add artifact metadata.
- Add provider-limit handling everywhere.

Phase 4: Feedback Memory

- Store rejection feedback.
- Convert repeated feedback into Suite rules.
- Show editable rules in Brand/Profile.

Phase 5: Campaign and Calendar Foundations

- CampaignDraft model.
- CalendarPlan/CalendarItem model.
- Draft-first UI.
- No automatic spending or publishing without approval.

Architectural Non-Negotiables

- Suite Memory is the heart.
- AI is a service layer, not UI logic.
- Generation is job-based.
- Publishing is separate from generation.
- Connections have granular readiness states.
- User feedback must become memory.
- Media must be public and durable for publishing.

- Mobile UX must be considered in every workflow.
- No silent zeros in analytics.
- No paid campaign launch without explicit confirmation.

Architect Questions

- هل نريد نبدأ بتقسيم `brand` JSON الآن، أم نكتفي بـ `SuiteContextBuilder` كطبقة توافق أولًا؟
- هل Railway worker service متاح عندك حاليًا، أم نحتاج queue DB-backed مؤقتًا؟
- هل campaign launch الحقيقي يدخل في أول نسخة مدفوعة، أم نؤجل للـ draft فقط؟
- هل لازم نبني audit log من الآن للنشر والحملات؟
- هل agencies تحتاج multi-user/team permissions قريبًا، أم لاحقًا؟

Recommendation

الخطوة المعمارية التالية يجب أن تكون صغيرة لكنها مركزية:

ابنِ `SuiteContextBuilder` وابدأ بنقل generation/onboarding/campaign prompts ليقرأوا منه بدل قراءة `suite.brand` مباشرة. هذا سيعطي المنتج عمود فقري واضح بدون refactor ضخم دفعة واحدة.

بعدها افصل onboarding UI إلى components. حاليًا هذه أكثر نقطة تسبب فوضى في التجربة والكود معًا.